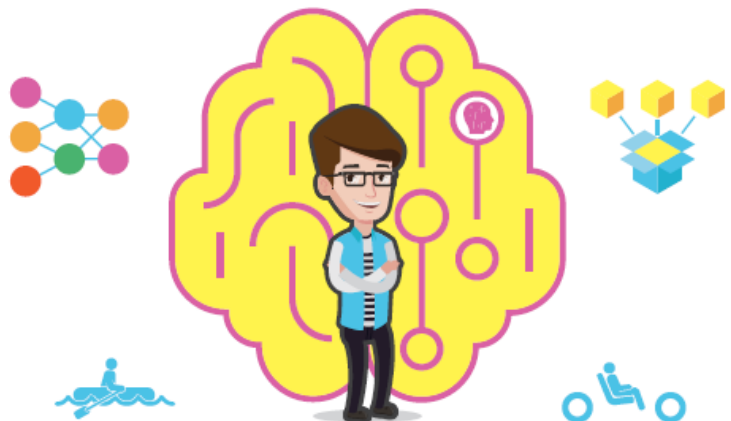


실습하며 배우는
딥러닝 입문
with **Kaggle**



타이타닉 생존자 예측부터 자율주행 자동차까지

[보조자료] 제공(www.booksr.co.kr)

장원두 지음

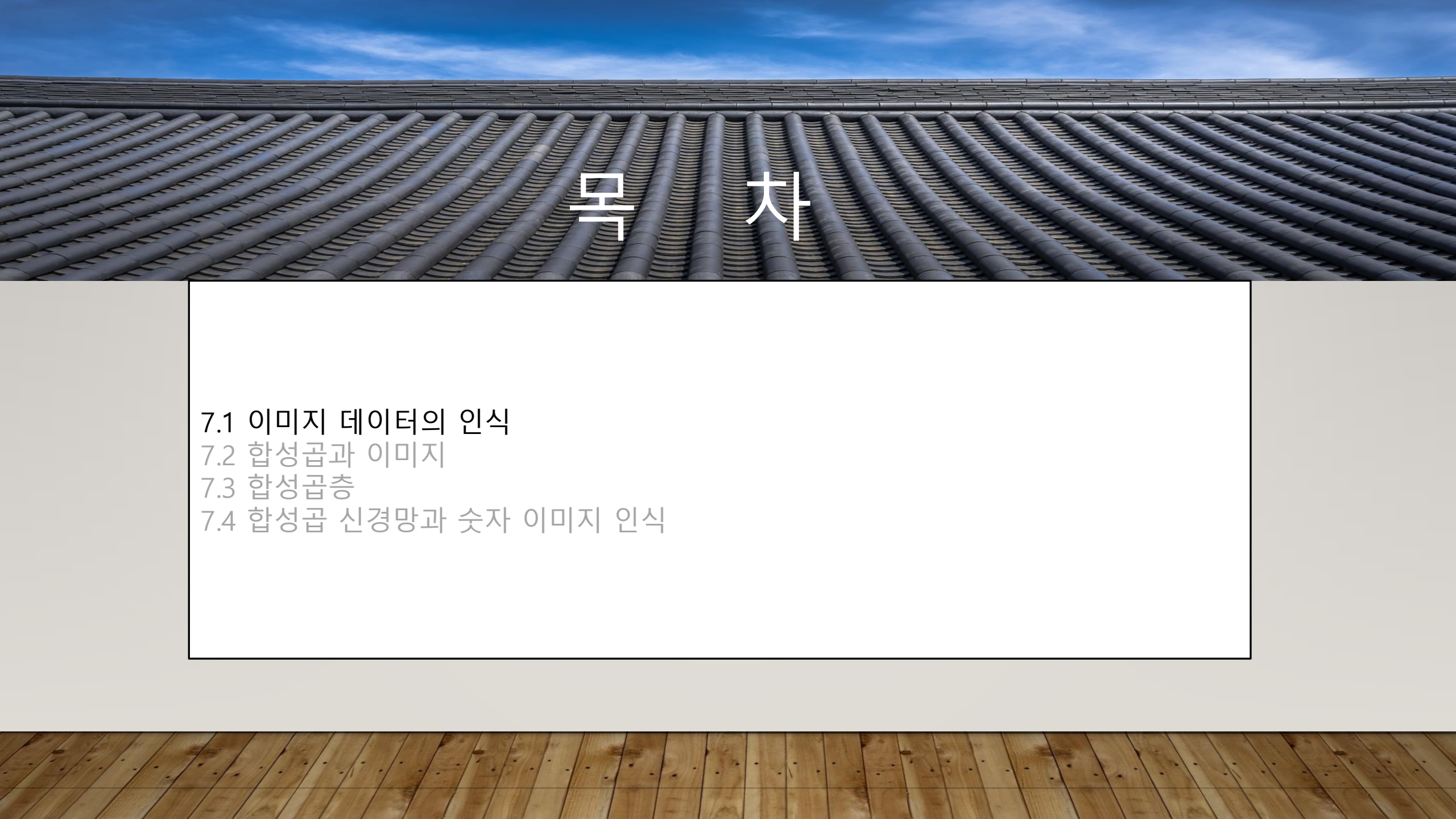
· 파이썬 소스 코드



실습하며 배우는

딥러닝 입문 WITH KAGGLE

7장: 이미지를 인식하는 네트워크, 합성곱 신경망



목 차

7.1 이미지 데이터의 인식

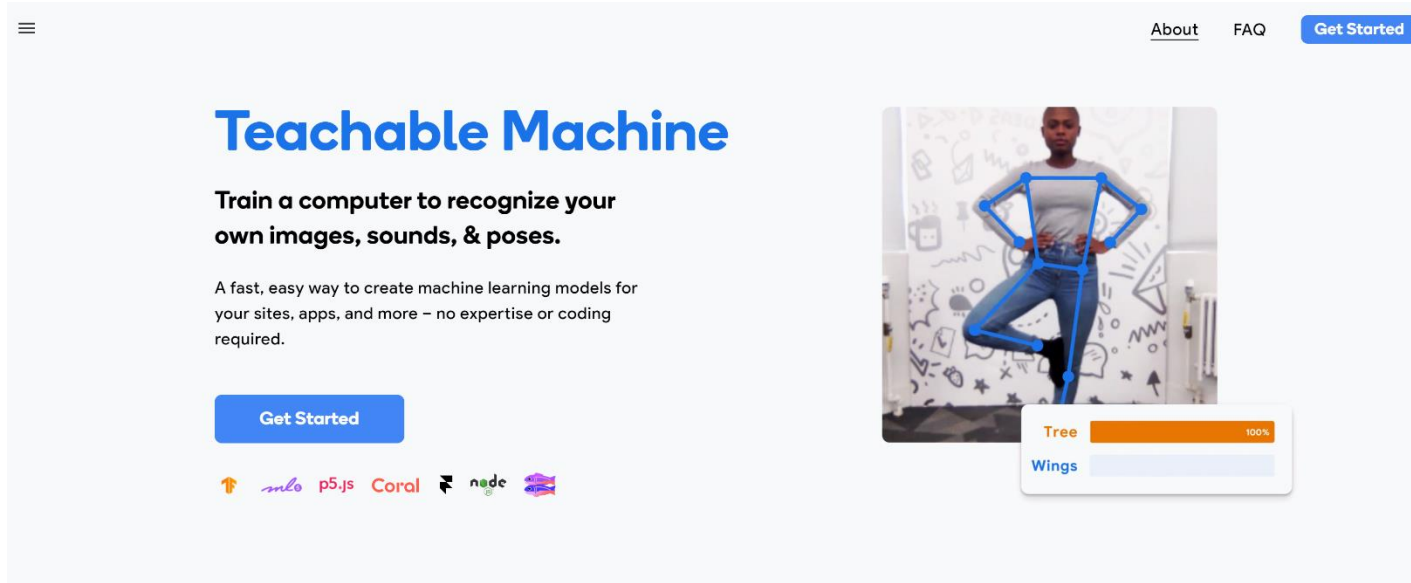
7.2 합성곱과 이미지

7.3 합성곱층

7.4 합성곱 신경망과 숫자 이미지 인식

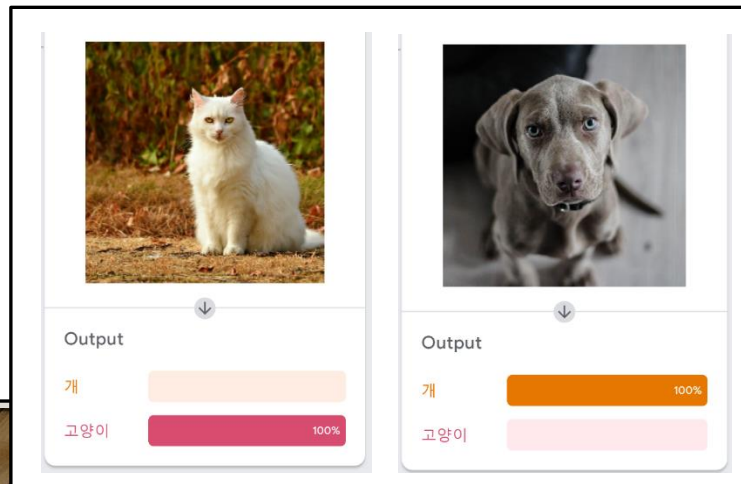


<https://teachablemachine.withgoogle.com/>



이미지 데이터 인식과 티쳐블 머신

- 이미지 데이터의 인식
 - 사진, 다양한 이미지를 분류하고 이해
- 티쳐블 머신
 - 별도의 코딩 작업 없이 드래그&드랍으로 딥러닝 모델을 생성할 수 있게 하는 웹 도구
 - 학습된 모델을 자바스크립트, python 등에서 활용 가능





이미지 데이터 인식과 티쳐블 머신

1. 티쳐블 머신 접속
2. 프로젝트 선택

- <https://teachablemachine.withgoogle.com/>
- Image Project 프로젝트 선택

New Project

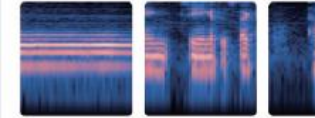
Open an existing project from Drive.

Open an existing project from a file.



Image Project

Teach based on images, from files or your webcam.



Audio Project

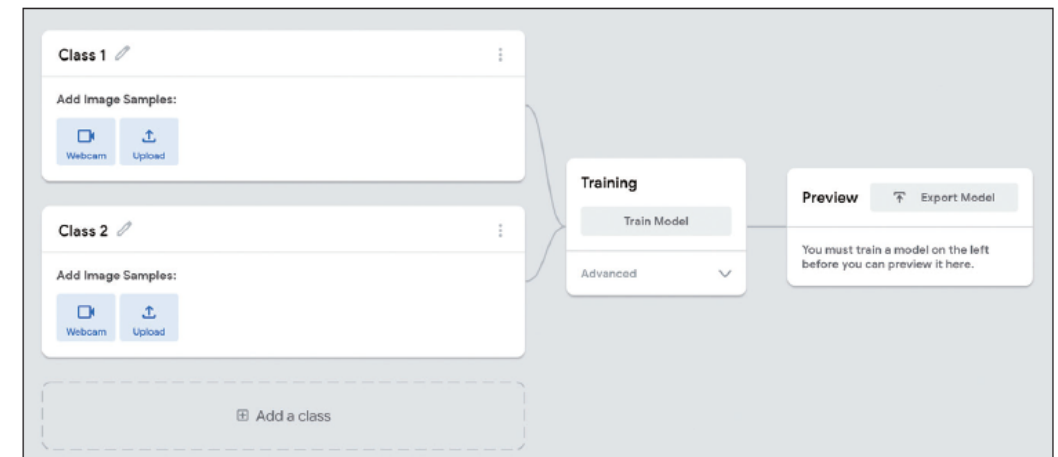
Teach based on one-second-long sounds, from files or your microphone.



Pose Project

Teach based on images, from files or your webcam.

[그림 7-2] 프로젝트 선택 화면



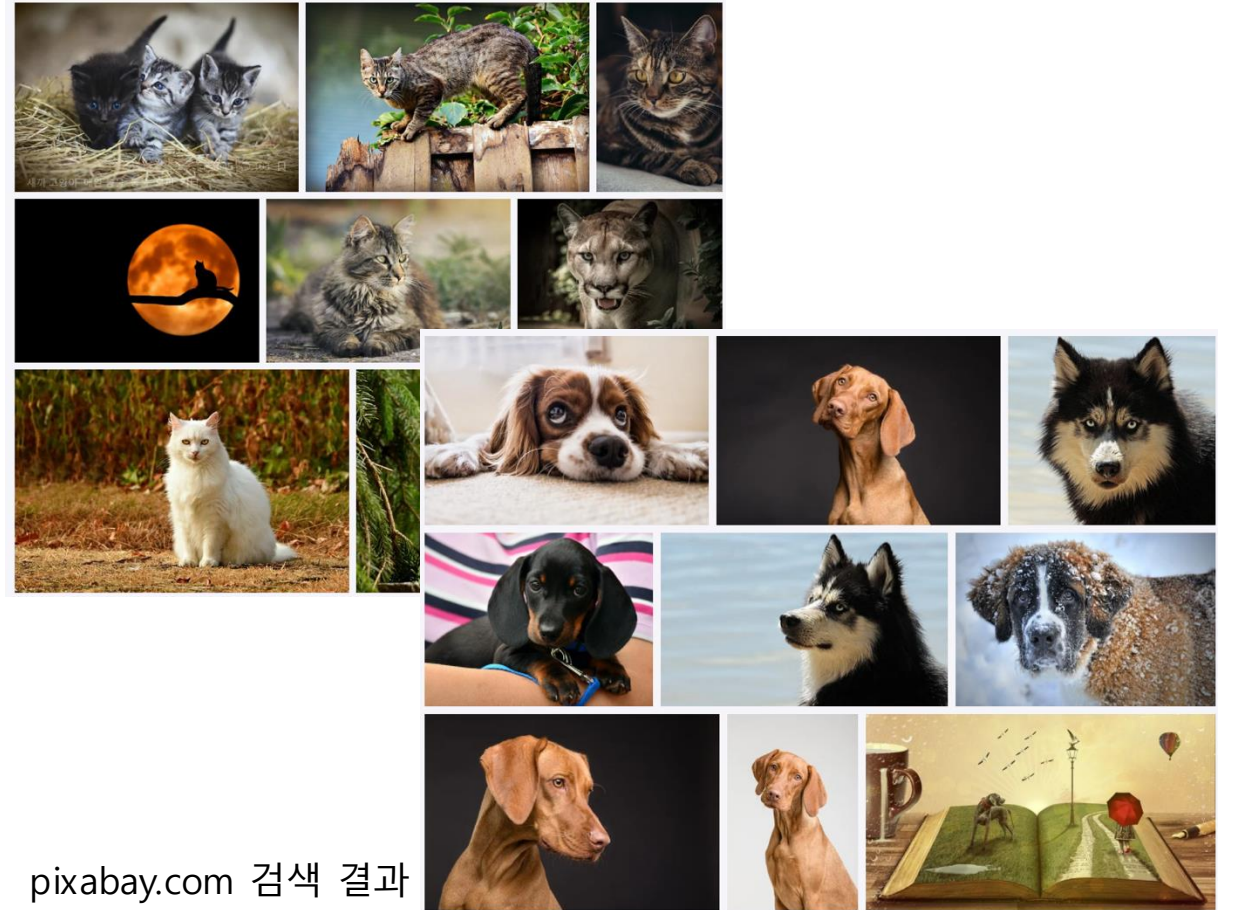
[그림 7-3] 이미지 프로젝트의 첫 화면



이미지 데이터 인식과 티쳐블 머신

1. 티쳐블 머신 접속
2. 프로젝트 선택
3. 이미지 다운로드

- <https://pixabay.com> 등에서 개/고양이 사진을 각각 10장씩 다운로드

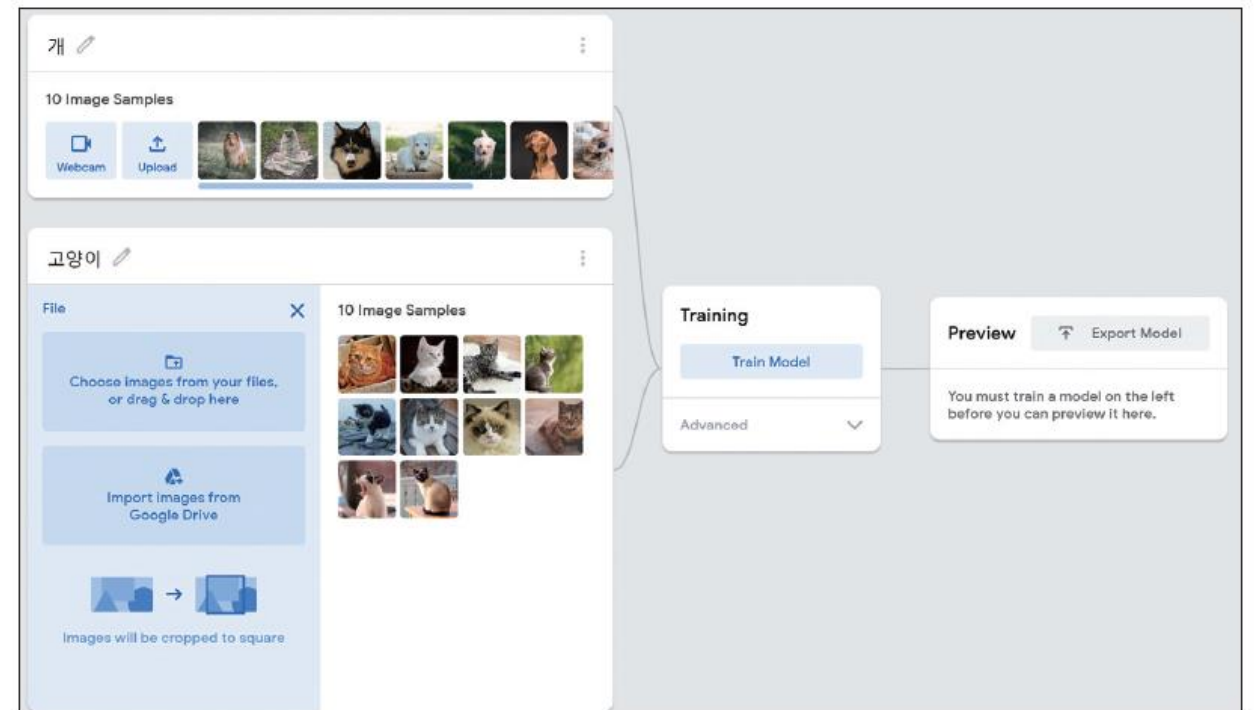




이미지 데이터 인식과 티쳐블 머신

1. 티쳐블 머신 접속
2. 프로젝트 선택
3. 이미지 다운로드
4. 티쳐블 머신에 이미지 업로드

- 티쳐블 머신 프로젝트에서 class1, class2의 이름을 각각 개, 고양이로 변경
- 클래스별로 이미지 업로드



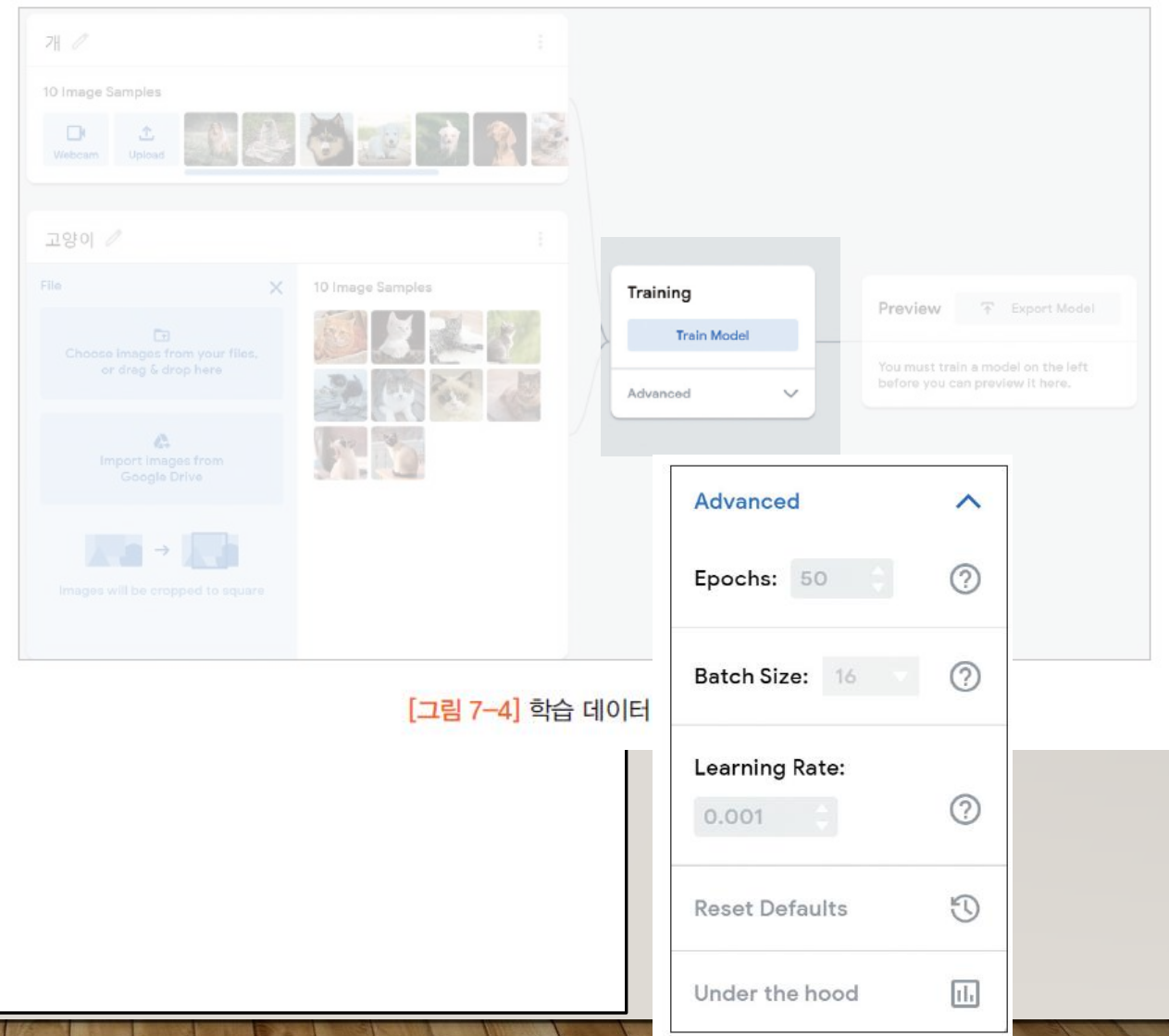
[그림 7-4] 학습 데이터 준비 화면



이미지 데이터 인식과 티쳐블 머신

1. 티쳐블 머신 접속
2. 프로젝트 선택
3. 이미지 다운로드
4. 티쳐블 머신에 이미지 업로드
5. 학습

- Train Model 버튼을 눌러 학습 시작
- 에포크 수, 배치 크기, 학습률 등 조정 가능



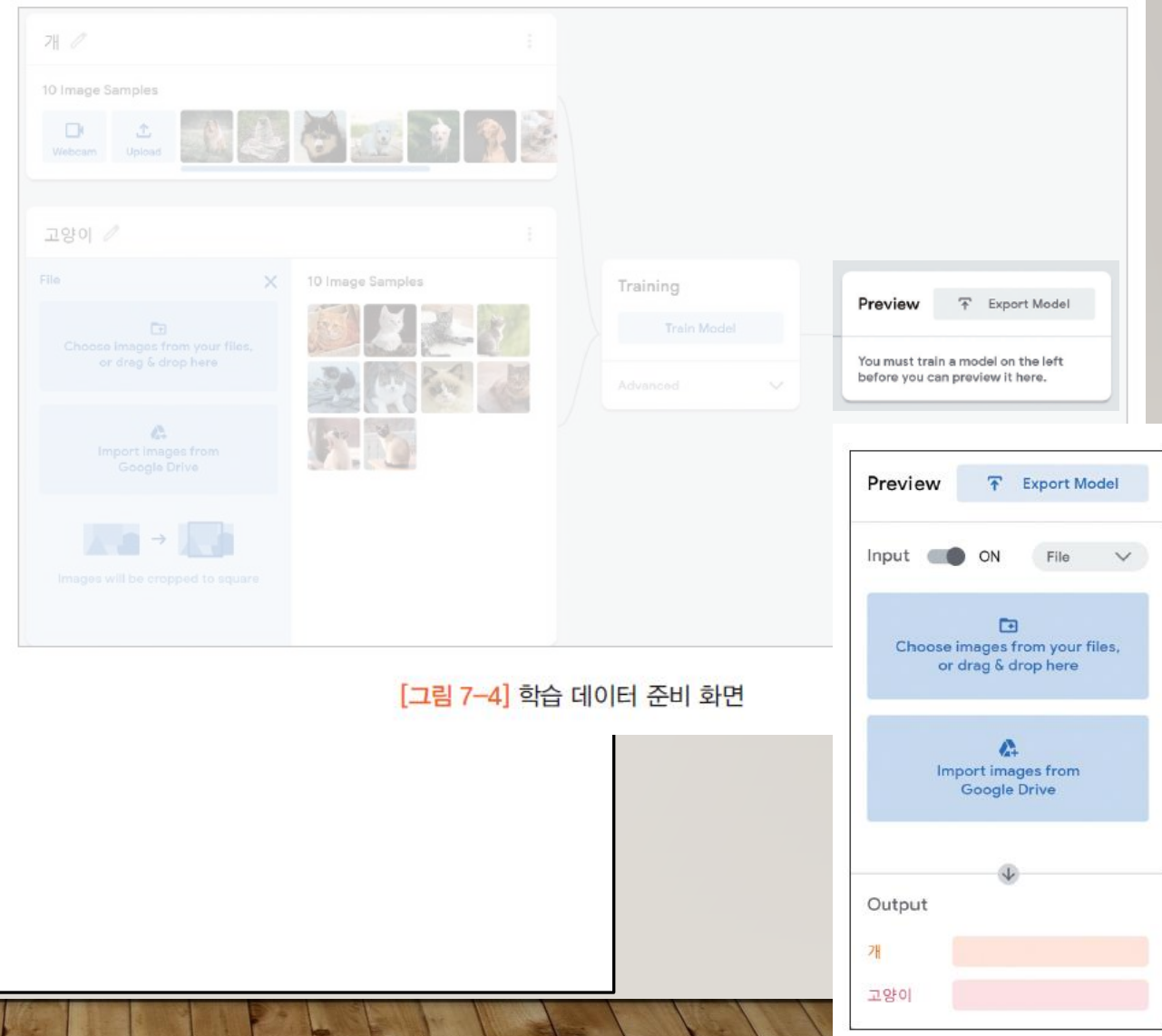
[그림 7-4] 학습 데이터



이미지 데이터 인식과 티쳐블 머신

1. 티쳐블 머신 접속
2. 프로젝트 선택
3. 이미지 다운로드
4. 티쳐블 머신에 이미지 업로드
5. 학습
6. 테스트 (1/2)

- 테스트를 위해 새로운 개/고양이 이미지 다운로드
- 학습이 완료되면 Preview 블록의 모양이 변경됨
- Input 토글 ON
- Webcam 에서 File 로 모드 변경
- 파일을 드래그&드랍으로 상자 안에 넣으면 결과 확인가능



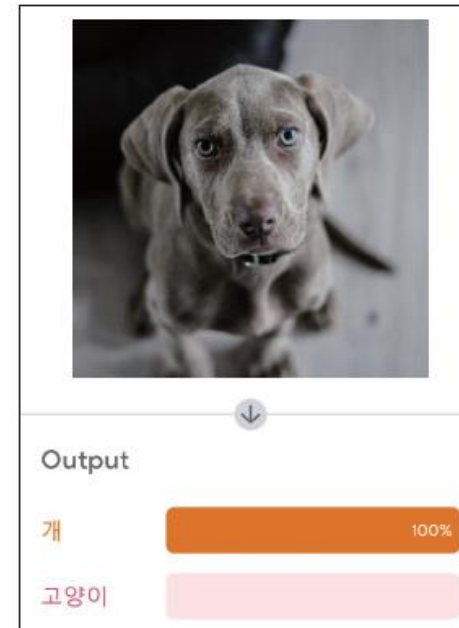
[그림 7-4] 학습 데이터 준비 화면



이미지 데이터 인식과 티쳐블 머신

1. 티쳐블 머신 접속
2. 프로젝트 선택
3. 이미지 다운로드
4. 티쳐블 머신에 이미지 업로드
5. 학습
6. 테스트 (2/2)

- 테스트를 위해 새로운 개/고양이 이미지 다운로드
- 학습이 완료되면 Preview 블록의 모양이 변경됨
- Input 토글 ON
- Webcam 에서 File 로 모드 변경
- 파일을 드래그&드랍으로 상자 안에 넣으면 결과 확인가능



[그림 7-6] 모델의 테스트 결과



추가 실습 (p.313)

1. 네 개 이상의 이미지(손가락 개수, 수화, 얼굴 구분 등)를 구분하는 자신만의 프로젝트를 만들어 테스트해 보시오. 핸드폰 등을 사용하여 사진을 촬영하여 테스트해 볼수 있다.

- 핸드폰/웹캠 등을 사용하여 손가락 개수 등의 이미지를 다수 촬영
- 촬영한 사진을 학습용/테스트용 데이터로 분리
- 학습용 사진을 티쳐블 머신에 업로드하여 학습
- 테스트용 사진을 사용하여 테스트
- 결과 분석 (정확도 계산)



목 차

- 7.1 이미지 데이터의 인식
 - 7.2 합성곱과 이미지
 - 7.3 합성곱층
 - 7.4 합성곱 신경망과 숫자 이미지 인식
- 



B ₀₀	B ₀₁	B ₀₂	B ₀₃	B ₀₄	B ₀₅
B ₁₀	B ₁₁	B ₁₂	B ₁₃	B ₁₄	B ₁₅
B ₂₀	B ₂₁	B ₂₂	B ₂₃	B ₂₄	B ₂₅
B ₃₀	B ₃₁	B ₃₂	B ₃₃	B ₃₄	B ₃₅
B ₄₀	B ₄₁	B ₄₂	B ₄₃	B ₄₄	B ₄₅
B ₅₀	B ₅₁	B ₅₂	B ₅₃	B ₅₄	B ₅₅



A ₀₀	A ₀₁	A ₀₂
A ₁₀	A ₁₁	A ₁₂
A ₂₀	A ₂₁	A ₂₂

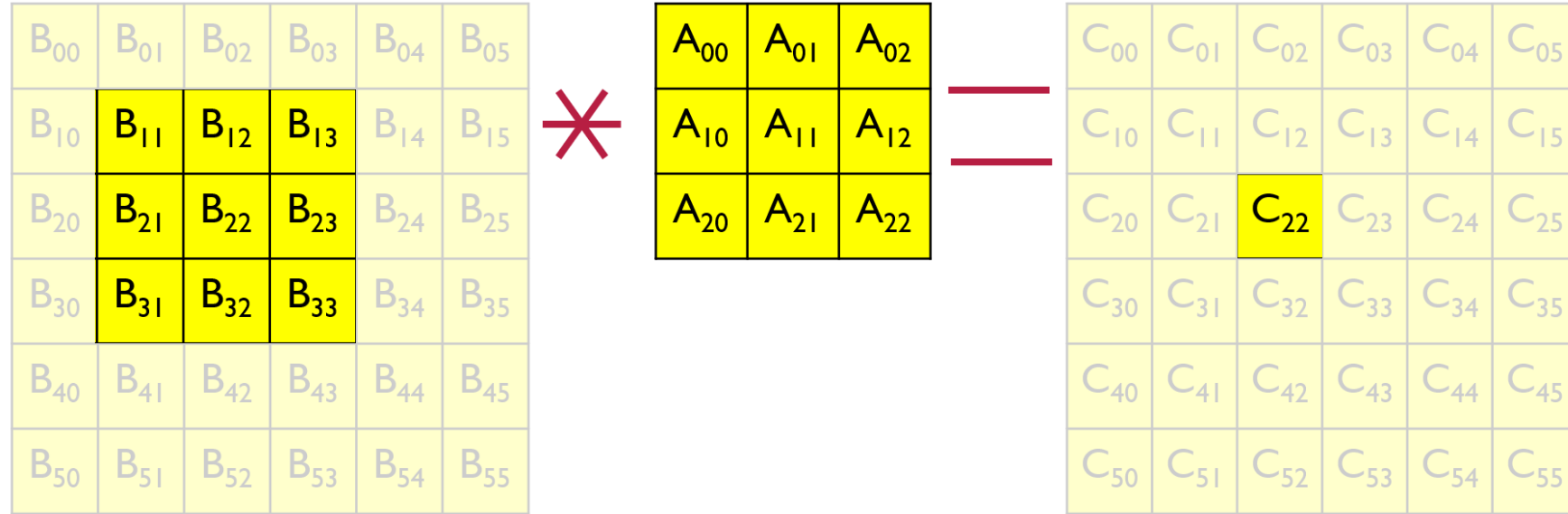


C ₀₀	C ₀₁	C ₀₂	C ₀₃	C ₀₄	C ₀₅
C ₁₀	C ₁₁	C ₁₂	C ₁₃	C ₁₄	C ₁₅
C ₂₀	C ₂₁	C ₂₂	C ₂₃	C ₂₄	C ₂₅
C ₃₀	C ₃₁	C ₃₂	C ₃₃	C ₃₄	C ₃₅
C ₄₀	C ₄₁	C ₄₂	C ₄₃	C ₄₄	C ₄₅
C ₅₀	C ₅₁	C ₅₂	C ₅₃	C ₅₄	C ₅₅

합성곱

- 두 행렬 사이의 연산
- 한 행렬을 고정시키고, 다른 행렬을 움직이며 겹쳐지는 부분의 원소들을 사용하여 계산
- 두 행렬의 값을 원소 단위로 곱하여 더함
- 딥러닝에서의 합성곱
 - flip을 뺀 단순화 버전

$$C_{xy} = \sum_{i=-1}^1 \sum_{j=-1}^1 A_{x+i,y+j} B_{x+i,y+j}$$



합성곱의 연산법

- 행렬 B 와 A 를 합성곱하여 새로운 행렬 C 가 생성된다고 할 때,
- C 의 특정 위치 C_{xy} 의 값은 다음 수식에 의해 결정

$$C_{xy} = \sum_{i=-1}^1 \sum_{j=-1}^1 A_{x+i,y+j} B_{x+i,y+j}$$

- 위 그림에서 C_{22} 의 경우,
- C_{22} 와 동일한 위치에 있는 B_{22} 를 중심으로 하여 행렬 A 와 동일한 크기의 행렬을 찾음
- $B_{11} \sim B_{33}$ 까지의 값과 A 행렬의 모든 값이 각각 위치별로 곱해진 후 합산됨



0	0	0	0	0	0	0	0
0	B ₀₀	B ₀₁	B ₀₂	B ₀₃	B ₀₄	B ₀₅	0
0	B ₁₀	B ₁₁	B ₁₂	B ₁₃	B ₁₄	B ₁₅	0
0	B ₂₀	B ₂₁	B ₂₂	B ₂₃	B ₂₄	B ₂₅	0
0	B ₃₀	B ₃₁	B ₃₂	B ₃₃	B ₃₄	B ₃₅	0
0	B ₄₀	B ₄₁	B ₄₂	B ₄₃	B ₄₄	B ₄₅	0
0	B ₅₀	B ₅₁	B ₅₂	B ₅₃	B ₅₄	B ₅₅	0
0	0	0	0	0	0	0	0



A ₀₀	A ₀₁	A ₀₂
A ₁₀	A ₁₁	A ₁₂
A ₂₀	A ₂₁	A ₂₂



C ₀₀	C ₀₁	C ₀₂	C ₀₃	C ₀₄	C ₀₅
C ₁₀	C ₁₁	C ₁₂	C ₁₃	C ₁₄	C ₁₅
C ₂₀	C ₂₁	C ₂₂	C ₂₃	C ₂₄	C ₂₅
C ₃₀	C ₃₁	C ₃₂	C ₃₃	C ₃₄	C ₃₅
C ₄₀	C ₄₁	C ₄₂	C ₄₃	C ₄₄	C ₄₅
C ₅₀	C ₅₁	C ₅₂	C ₅₃	C ₅₄	C ₅₅

제로-패딩

- 위 그림에서 C₀₀과 같이 가장자리의 값을 계산하는 경우, B행렬에 주변값이 없으므로, 행렬 B에 값이 0으로 된 패드(Pad)가 존재한다고 가정
- 값이 없는 경우 0을 곱하여 합산함. 0과의 곱은 모두 0이므로, A행렬에서 해당 위치의 값은 무시됨
- 위 그림의 경우

$$C_{00} = A_{11}B_{00} + A_{12}B_{01} + A_{21}B_{10} + A_{22}B_{11}$$



*

1	0	-1
2	0	-2
1	0	-1



합성곱과 이미지

- 합성곱은 컴퓨터 비전에서 영상 특징 추출 등에 널리 쓰이는 연산임
- 이미지를 하나의 매트릭스로 생각하면, 작은 매트릭스 (3x3)의 합성곱 연산을 통해 에지 추출, 노이즈 제거 등이 가능
- 딥러닝에서 에지 등과 같은 이미지의 특징을 추출하는 데에 활용됨



```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread('bird.jpg', cv2.IMREAD_GRAYSCALE)
cv2_imshow(img)
```

→ 파일이름 변경



[그림 7-10] 코랩에서 불러와 화면에 표시한 이미지

합성곱 실습

- 영상처리에 사용되는 opencv 라이브러리를 사용하여 합성곱 실습
- 인터넷에서 적당한 이미지를 다운로드 받고 코랩에 업로드
- cv2
 - opencv 라이브러리
- cv_imshow
 - 코랩을 위해 디자인된 함수
 - 코랩 결과창에 이미지 표시
- imread(...)
 - 이미지를 읽어옴
- IMREAD_GRAYSCALE
 - 그레이스케일로 영상을 읽어옴



```
import numpy as np
filter = np.array([ [-1,0,1],
                    [-2,0,2],
                    [-1,0,1]])
f_img = cv2.filter2D(img, -1, filter)
cv2_imshow(f_img)
```

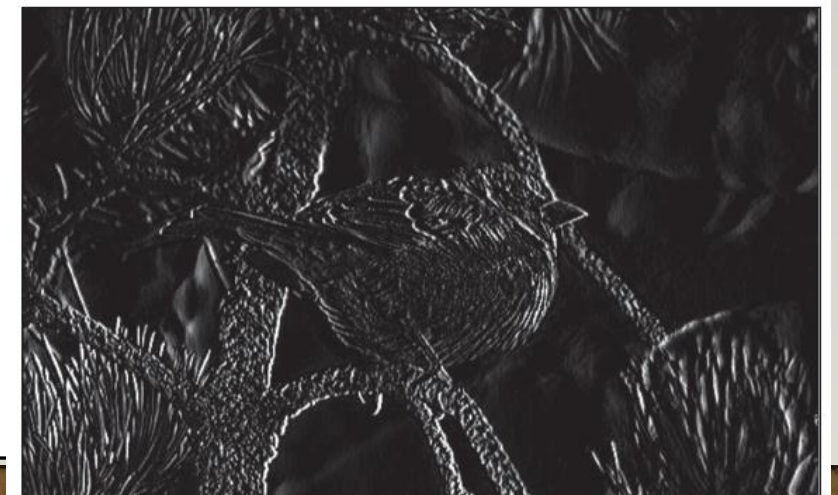
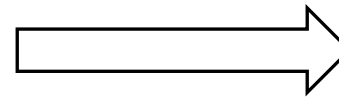


[그림 7-10] 코랩에서 불러와 화면에 표시한 이미지

*

-1	0	1
-2	0	2
-1	0	1

필터



[그림 7-11] 합성곱 연산이 수행된 결과 이미지

합성곱 실습

- 그레이스케일 이미지
 - 2차원 배열로 저장됨
- 필터
 - 원본 이미지에 대해 합성곱을 수행할 행렬
- 원본 이미지와 필터간에 합성곱 수행
 - filter2D([이미지 배열], -1, [필터 배열])
 - 두 번째 파라미터는 출력 영상 타입.
 - -1은 원본 이미지와 동일한 타입으로 출력
- 필터 종류에 따라 다른 결과 획득
 - 아래 그림의 필터는 윤곽선을 강조하는 소벨 필터

이미지



추가 실습 (p.313)

- 다음 필터를 사용하여 원본 이미지에 합성곱을 수행하라

$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$

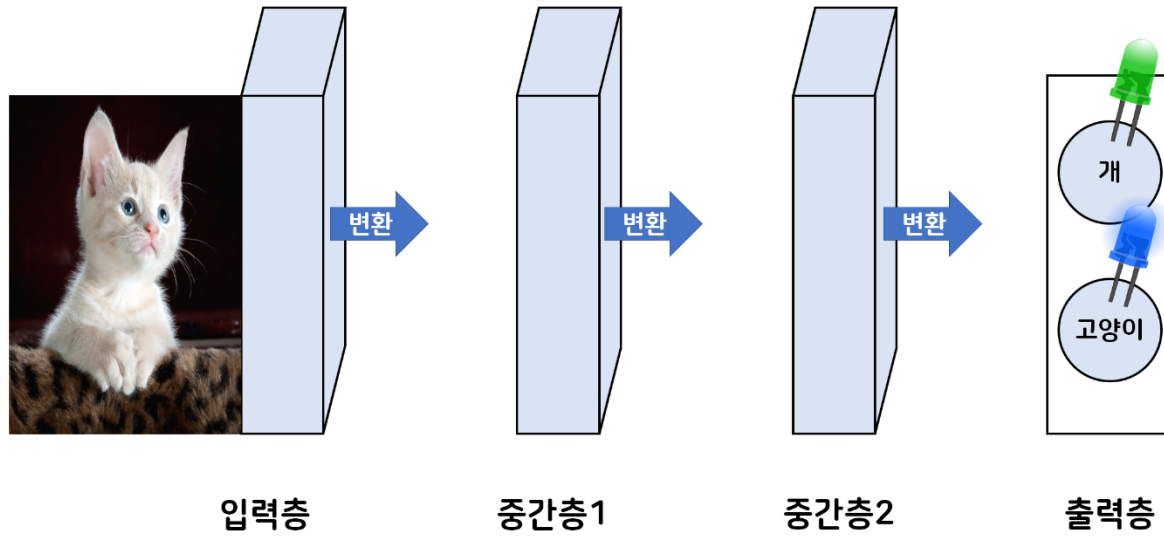
$\frac{1}{16}$	$\frac{1}{8}$	$\frac{1}{16}$
$\frac{1}{8}$	$\frac{1}{4}$	$\frac{1}{8}$
$\frac{1}{16}$	$\frac{1}{8}$	$\frac{1}{16}$

-1	-1	-1
-1	8	-1
-1	-1	-1



목 차

- 7.1 이미지 데이터의 인식
- 7.2 합성곱과 이미지
- 7.3 합성곱층**
- 7.4 합성곱 신경망과 숫자 이미지 인식

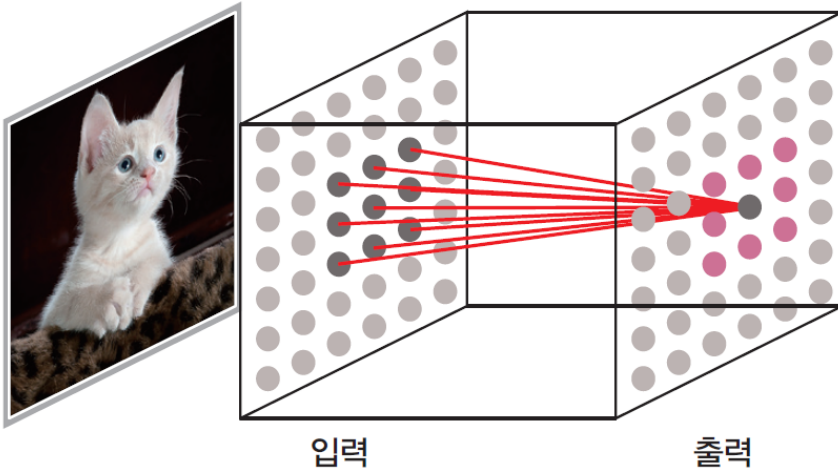


딥러닝 네트워크의 기본구조

- 딥러닝 모델은 입력층과 출력층 사이를 이어주는 여러 개의 네트워크 층으로 구성됨
- 네트워크 층에는 다양한 종류가 있음
 - e.g. 완전연결층, 합성곱층

합성곱 층

- 합성곱 연산이 이루어지는 층
- 각 층의 역할은 "받은 입력을 변환하여 출력하는 것"
- 이미지를 효과적으로 학습하기 위한 모델
- 이미지의 특성 (에지 등)을 효과적으로 학습함
- 알파고의 경우 바둑판을 0,1,2 의 값을 가지는 이미지로 바꾸어 활용



[그림 7-13] 합성곱층의 내부구조

합성곱 과 합성곱

층 필터와 가중치

- 합성곱에 사용하는 필터는 노드 사이를 연결하는 에지로 표현
- edge의 값 (가중치)는 같은 층 안에서 서로 공유됨
- e.g. 3x3필터를 사용하는 합성곱 층은 가중치의 개수가 9개

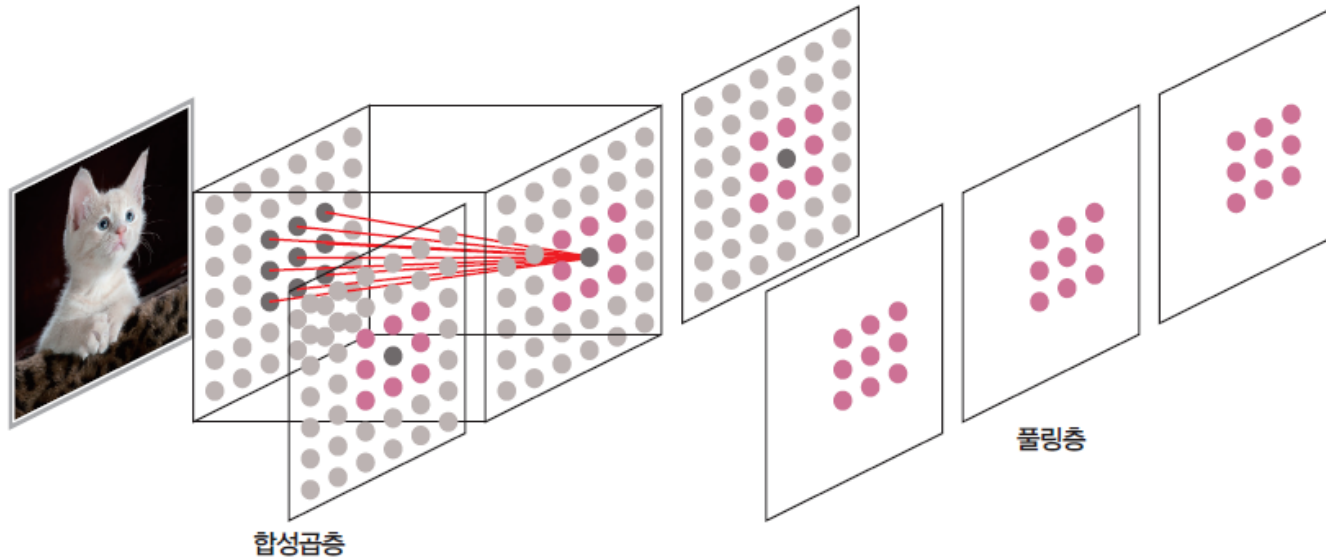
합성곱 층의 연결

- 임의의 한 노드는 이전 층의 일부 노드와 연결
 - 모든 노드와 연결되는 완전연결층과 대조
 - 이전 층에 있는 노드 중 인접한 위치에 있는 노드들과 연결됨
 - 6x7이미지에 3x3필터를 사용하는 경우
전체 edge의 개수는 $(6 \times 7) \times (3 \times 3)$

-1	0	1
-2	0	2
-1	0	1

== 가중치

필터



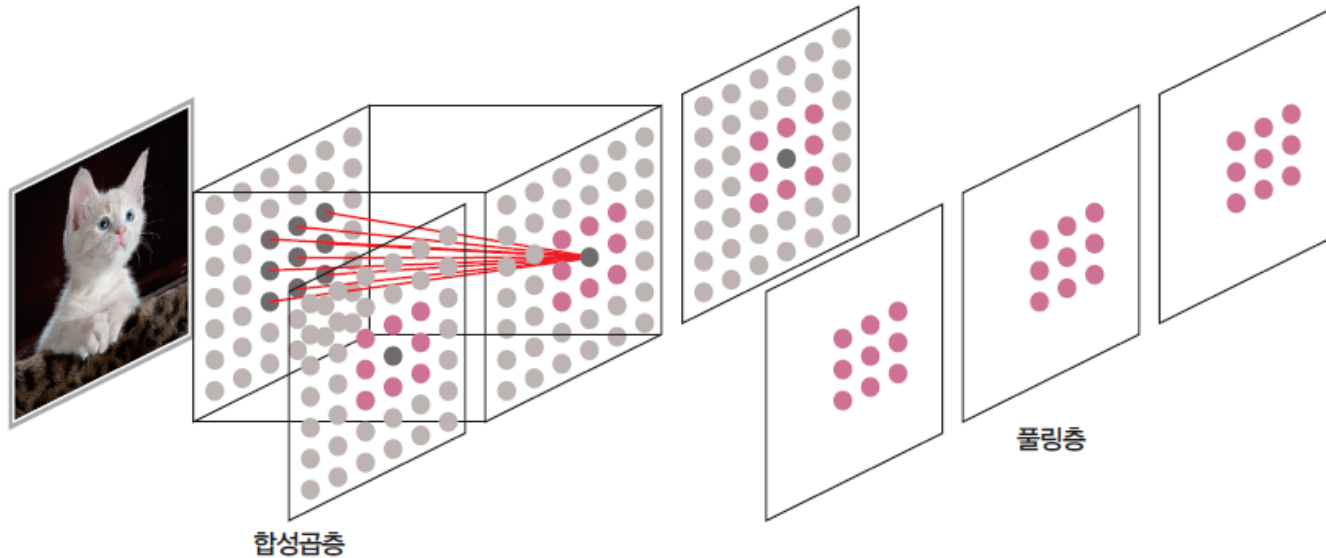
[그림 7-14] 다수 필터를 사용하는 합성곱층과 풀링층

다수 필터를 사용하는 합성곱 층

- 합성곱 층의 목표: 중요한 특징을 추출하는 최적의 가중치를 찾는 것
- 다양한 특징을 추출하기 위해 합성곱 층에 여러 개의 필터를 사용하는 것이 일반적
- 필터의 개수에 따라 데이터의 크기가 증가
- 가중치 개수 = [필터 개수] × [필터너비] × [필터높이]

합성곱 층의 학습

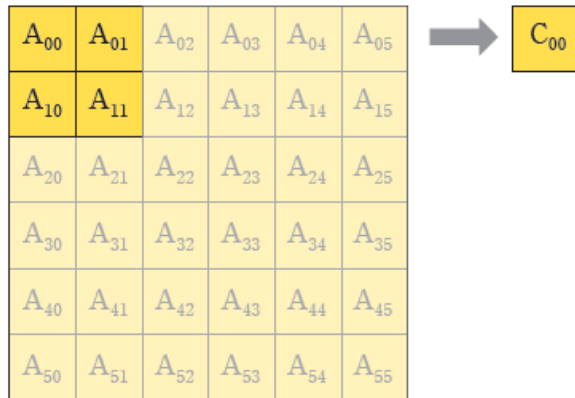
- 가중치의 개수는 필터 개수에 비례하여 증가
- 합성곱 층, 이후 풀링층을 사용하여 크기를 절반 정도로 줄이는 것이 일반적
- 풀링층은 resize, subsampling 등으로 이해할 수 있으며
 - MaxPooling, MinPooling 등이 있음



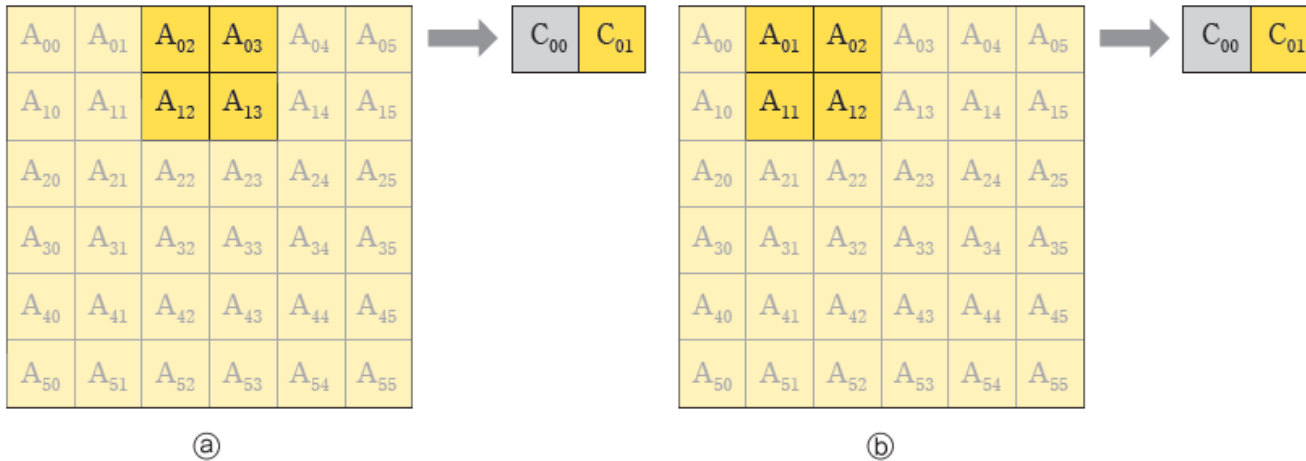
[그림 7-14] 다수 필터를 사용하는 합성곱층과 풀링층

풀링층

- 합성곱을 통해 추출된 특징의 크기를 감소
 - 문제를 단순화시킴으로 학습 난이도가 감소
- 특정 범위 내에서 대표하는 값을 선택/계산
 - 범위 내의 최대/최소/평균 등을 선택하여 해당 특징들을 대표시킴
 - MaxPooling: 최대값을 선택
 - MinPooling: 최솟값을 선택
 - AveragePooling: 평균값을 선택
 - 합성곱이 수행된 이미지의 경우
컬러값이 0에 가까울수록 정보가 적은 경우가 많으므로 (에지 이미지 등) MaxPooling을 사용하는 것이 일반적



[그림 7-15] 풀링



[그림 7-16] Stride에 따른 풀링의 차이 - ① stride=2 ② stride=1

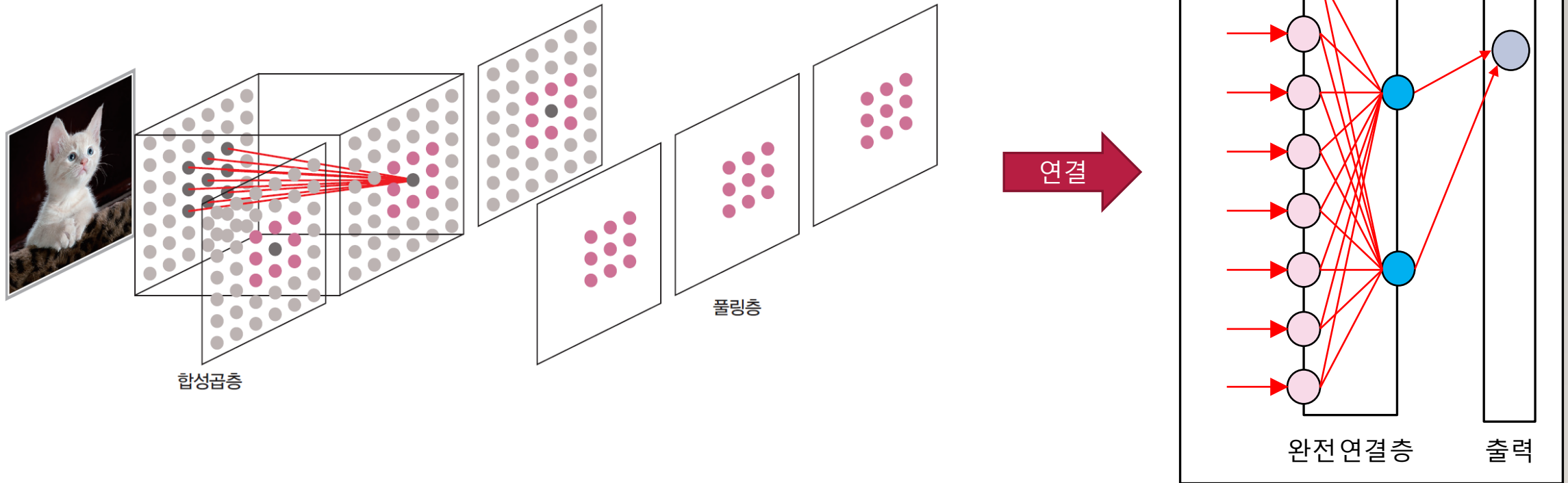
책에 (a), (b)의 stride크기가 서로 바뀌어
있음

풀링의 예

- Step 1 (풀링): [그림 7-15]
 - 풀 크기(pool_size)는 2x2, MaxPooling인 경우
 - 풀 윈도우 (pool window)는 A_{00} , A_{01} , A_{10} , A_{11}
 - 풀 윈도우의 값 중 최댓값이 선택되어 C_{00} 의 값으로 지정됨
- Step 2: [그림 7-16]
 - stride가 2인 경우, 풀 윈도우가 2 칸 이동 (a)
 - stride가 1인 경우, 풀 윈도우가 1 칸 이동 (b)
 - 이동한 지점에서 풀링 수행
- Step 2를 계속 반복
 - x축으로 끝까지 이동한 경우 x축의 처음으로 돌아가고, y축으로 stride의 크기에 따라 이동
- stride=2, pool_size=2인 경우, 풀링 후의 크기는 입력 이미지에 비해 $\frac{1}{2}$ 로 감소



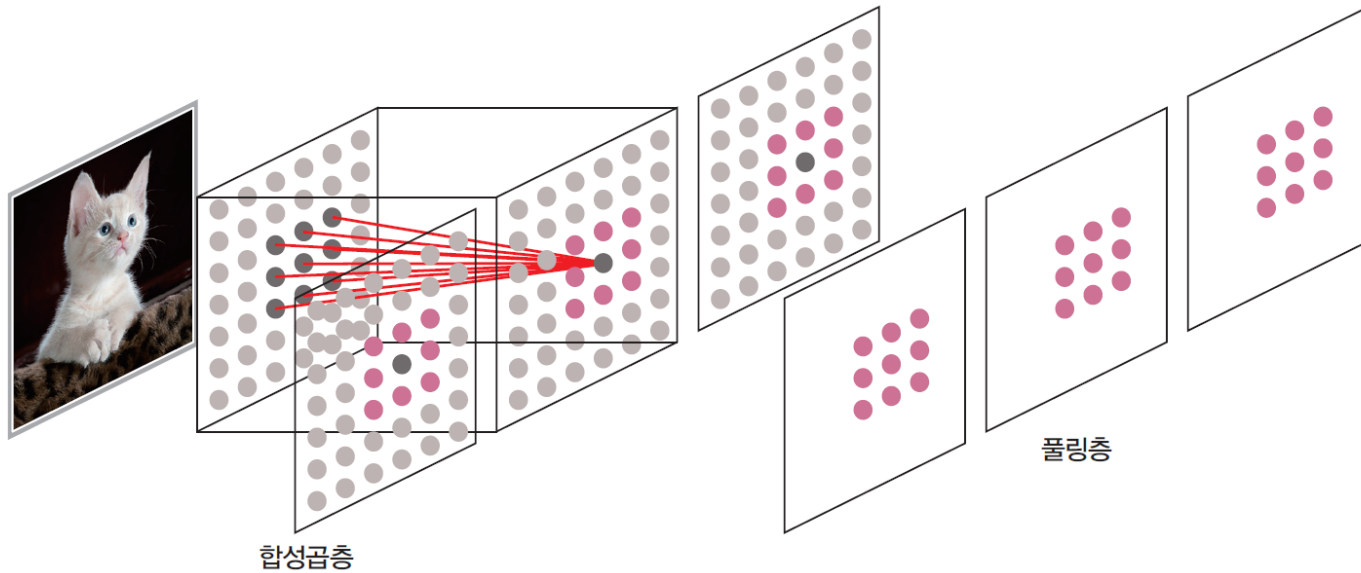
합성곱 층과 완전연결층의 연결 (1/3)



- 합성곱 층의 출력은 2/3차원의 특징 데이터
- 합성곱 층의 출력에서 곧바로 데이터의 분류/인식 등을 하는 것은 어려움
- 합성곱 층과 완전연결층을 연결하여 이미지 인식 네트워크를 만드는 것이 보편적
- 합성곱 신경망 (Convolutional Neural Network: CNN)
 - 합성곱 층과 완전연결층이 연결된 딥러닝 네트워크

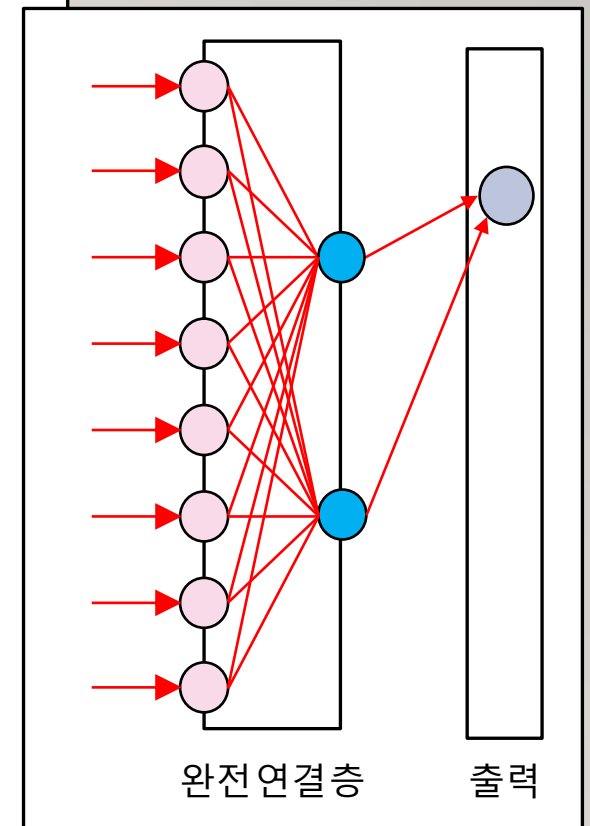


합성곱 층과 완전연결층의 연결 (2/3)



FLATTENING 층

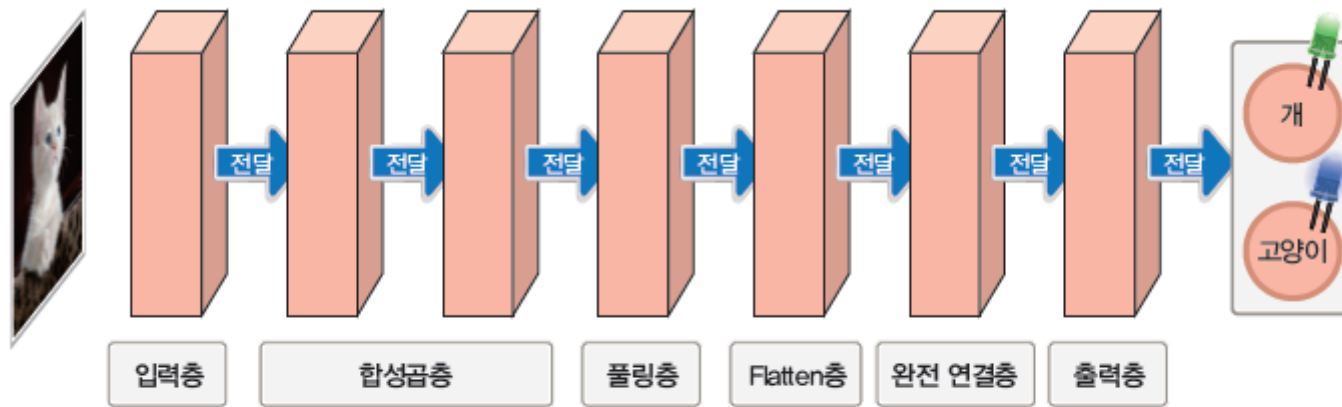
연결



- 합성곱 층의 출력은 3차원 구조이므로, 이를 1차원 형태로 바꾸어 주는 과정 필요
- 데이터를 1차원으로 바꾸어 주는 Flattening 층을 사용하여 합성곱 층과 완전연결층을 연결



합성곱 층과 완전연결층의 연결 (3/3)



[그림 7-17] 합성곱 신경망의 예

합성곱 층을 활용하는 네트워크

- 합성곱층은 풀링층, Flatten층, 완전연결층과 함께 연결되어 사용됨
- 합성곱층과 풀링층의 비율은 입력층의 크기 등을 고려하여 조정될 수 있음
- [그림 7-17]에서 입력층과 출력층은 개념적으로 존재하는 층 (코드 구현에서는 사용되지 않음)
- 합성곱 네트워크에서 조정 가능한 파라미터
 - 각 층의 개수 및 구조
 - 합성곱층에서의 필터 수, 필터의 크기
 - 풀링층에서 풀 크기, stride
 - 출력과 직접 연결되지 않은 완전연결층에서 출력 노드의 수



목 차

- 7.1 이미지 데이터의 인식
 - 7.2 합성곱과 이미지
 - 7.3 합성곱층
 - 7.4 합성곱 신경망과 숫자 이미지 인식
- 



MNIST 데이터셋

- 미국 국립표준연구소(NIST)에서 제작한 숫자 필기 데이터의 수정본
- 데이터 구성
 - 28x28그레이스케일 이미지
 - 학습 데이터 6만개
 - 테스트 데이터 1만 개
 - 참여자: 고등학생, 미국 인구조사국 직원
- 코랩의 MNIST 데이터셋
 - 학습 데이터 2만개
 - 테스트 데이터 1만개



실습

1. 데이터 읽기



```
import pandas as pd
import numpy as np
data = pd.read_csv('./sample_data/mnist_train_small.csv', header=None)
data = np.array(data)
```



```
print(data)
print(data.shape)
```



```
[[6 0 0 ... 0 0 0]
 [5 0 0 ... 0 0 0]
 [7 0 0 ... 0 0 0]
 ...
 [2 0 0 ... 0 0 0]
 [9 0 0 ... 0 0 0]
 [5 0 0 ... 0 0 0]]
(20000, 785)
```

[그림 7-18] MNIST 데이터

- pandas와 numpy 라이브러리 사용
- read_csv 함수를 사용하여 csv 파일 읽기
- csv 파일에 열 이름이 따로 없으므로, header=None 옵션 지정
- array 함수를 사용하여 데이터 변환



실습

1. 데이터 읽기
2. 데이터 확인

- print 함수를 사용하여 데이터 확인
- 첫 번째 데이터의 모양을 확인
 - 첫 번째 열은 숫자 번호
 - 두 번째부터 마지막 열까지는 28x 28 크기의 이미지 컬러 정보가 1차원으로 저장되어 있음

```
print(data)
print(data.shape)
```

```
print(data[0,:])
```

```
print(data)
print(data.shape)
```

$$\begin{bmatrix} 6 & 0 & 0 & \dots & 0 & 0 & 0 \\ 5 & 0 & 0 & \dots & 0 & 0 & 0 \\ 7 & 0 & 0 & \dots & 0 & 0 & 0 \\ \vdots & & & & & & \\ 2 & 0 & 0 & \dots & 0 & 0 & 0 \\ 9 & 0 & 0 & \dots & 0 & 0 & 0 \\ 5 & 0 & 0 & \dots & 0 & 0 & 0 \end{bmatrix}$$

(20000, 785)

[그림 7-18] MNIST 데이터

```
print(data[0,:])
```

```
array([ 6,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0, 24, 67, 67, 18,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0, 131, 252, 252, 66,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 159, 250, 232, 30,
       32,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 15, 222, 252])
```

[그림 7-19] MNIST 데이터셋 첫 번째 데이터



실습

1. 데이터 읽기
2. 데이터 확인
3. 이미지 데이터와 정답 정보 분할

- 슬라이싱을 사용하여 데이터 분할
 - 1번 열부터 마지막 열까지를 `x_train`에 저장
 - 0번 열을 `y_train`에 저장
- 크기 변경
 - `x_train`은 `reshape` 함수를 사용하여 `nx28x28` 크기로 변경
- `reshape` 함수의 파라미터
 - -1은 컴퓨터가 자동으로 크기를 계산하도록 하는 옵션
 - 크기를 지정할 때 1개 차원은 -1로 지정할 수 있음



```
x_train = data[:,1:]  
y_train = data[:,0]  
x_train = x_train.reshape(-1, 28, 28)
```



```
x_train.shape
```



```
(20000, 28, 28)
```

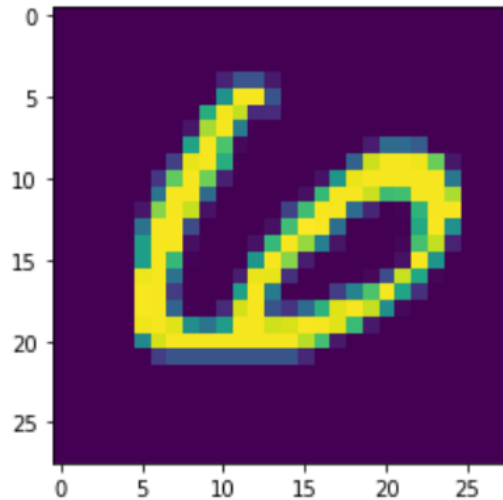



실습

1. 데이터 읽기
2. 데이터 확인
3. 이미지 데이터와 정답 정보 분할
4. 이미지 데이터의 확인



```
import matplotlib.pyplot as plt  
plt.imshow(x_train[0,:,:])
```



• 이미지 데이터의 확인

- matplotlib을 사용하여 데이터를 이미지 형태로 화면에 출력 가능
- x_train의 첫 번째 데이터를 화면에 출력

• 정답 확인

- y_train의 첫 번째 값을 출력하여 해당 이미지의 정답(레이블)을 확인



```
y_train[0]
```



```
6
```



실습

1. 데이터 읽기
2. 데이터 확인
3. 이미지 데이터와 정답 정보 분할
4. 이미지 데이터의 확인
5. 데이터 차원변경, 수치화



```
import tensorflow.keras as keras
from keras.models import Sequential
from keras import layers
from keras import optimizers
```



```
x_train = np.expand_dims(x_train,-1)
y_train = keras.utils.to_categorical(y_train)
```



```
y_train[0,:]
```



```
array([0., 0., 0., 0., 0., 0., 1., 0., 0., 0.], dtype=float32)
```

- 이미지 데이터의 확인
 - 이미지 데이터를 딥러닝의 학습에 사용하기 위해서는 차원을 1개 추가해야 함
 - 학습 데이터를 $n \times 28 \times 28 \times 1$ 크기로 변경 (expand_dims 함수)
- 원-핫 인코딩
 - 0~9의 값을 가지는 y_train을 원-핫 인코딩
 - 0 또는 1의 값을 가지는 열의 크기가 10인 배열로 변경 ($n \times 10$)
- 결과 확인
 - 바뀐 모양과 값을 코드창에서 확인



```
x_train.shape
```



```
(20000, 28, 28, 1)
```



```
y_train.shape
```



```
(20000, 10)
```



실습

1. 데이터 읽기
2. 데이터 확인
3. 이미지 데이터와 정답 정보 분할
4. 이미지 데이터의 확인
5. 데이터 차원변경, 수치화
6. 딥러닝 모델 생성

• 딥러닝 모델

- Sequential 모델
- 합성곱 층 2개, MaxPooling 층 1개, Flatten층, 완전연결층으로 구성
- 합성곱 층의 필터 크기는 3, 필터 개수는 각각 8, 16으로 지정
- Maxpooling 층의 strides는 2로 지정, 풀 크기는 기본값(2)로 지정
- 완전연결층의 노드 수는 10개로 지정
- 분류 네트워크이므로 손실 계산은 categorical_crossentropy로 설정
- 최적화 도구는 RMSprop, 학습률은 0.001로 설정
- 최적화 도구와 학습률 등은 변경 가능



```
model = Sequential()
model.add(layers.Conv2D(filters=8, kernel_size = 3, input_shape=(28,28,1), activation='relu'))
model.add(layers.Conv2D(filters=16, kernel_size = 3, activation='relu'))
model.add(layers.MaxPooling2D(strides= 2, padding='same'))
model.add(layers.Flatten())
model.add(layers.Dense(units=10, activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer=optimizers.RMSprop(lr=0.001), metrics=['accuracy'])
```




실습

1. 데이터 읽기
2. 데이터 확인
3. 이미지 데이터와 정답 정보 분할
4. 이미지 데이터의 확인
5. 데이터 차원변경, 수치화
6. 딥러닝 모델 생성
7. 학습

- 학습 파라미터
 - 에포크 10회
 - 배치 크기 64
 - 검증 데이터 분할 0.3
(전체 학습 데이터에서 30%는 검증용으로, 나머지는 학습용으로 사용)



```
model.fit(x_train,y_train,epochs= 10, batch_size=64, validation_split=0.3)
```

```
Epoch 1/10
219/219 [=====] - 10s 40ms/step - loss: 9.4353 - accuracy: 0.7072 - val_loss: 0.2582 - val_accuracy: 0.9423
Epoch 2/10
219/219 [=====] - 8s 38ms/step - loss: 0.1977 - accuracy: 0.9535 - val_loss: 0.1816 - val_accuracy: 0.9522
Epoch 3/10
219/219 [=====] - 8s 38ms/step - loss: 0.0780 - accuracy: 0.9768 - val_loss: 0.1517 - val_accuracy: 0.9655
Epoch 4/10
219/219 [=====] - 8s 39ms/step - loss: 0.0449 - accuracy: 0.9869 - val_loss: 0.1750 - val_accuracy: 0.9675
Epoch 5/10
219/219 [=====] - 8s 39ms/step - loss: 0.0330 - accuracy: 0.9903 - val_loss: 0.1459 - val_accuracy: 0.9690
Epoch 6/10
219/219 [=====] - 8s 38ms/step - loss: 0.0193 - accuracy: 0.9933 - val_loss: 0.1411 - val_accuracy: 0.9770
Epoch 7/10
219/219 [=====] - 8s 39ms/step - loss: 0.0157 - accuracy: 0.9949 - val_loss: 0.1388 - val_accuracy: 0.9768
Epoch 8/10
219/219 [=====] - 8s 38ms/step - loss: 0.0113 - accuracy: 0.9967 - val_loss: 0.1492 - val_accuracy: 0.9770
Epoch 9/10
219/219 [=====] - 8s 38ms/step - loss: 0.0073 - accuracy: 0.9975 - val_loss: 0.1905 - val_accuracy: 0.9748
Epoch 10/10
219/219 [=====] - 8s 38ms/step - loss: 0.0067 - accuracy: 0.9978 - val_loss: 0.1742 - val_accuracy: 0.9747
```

[그림 7-21] MNIST 데이터의 학습 화면



실습

1. 데이터 읽기
2. 데이터 확인
3. 이미지 데이터와 정답 정보 분할
4. 이미지 데이터의 확인
5. 데이터 차원변경, 수치화
6. 딥러닝 모델 생성
7. 학습
8. 테스트 데이터 읽기

- pandas와 numpy 라이브러리 사용
- read_csv 함수를 사용하여 csv 파일 읽기
- csv 파일에 열 이름이 따로 없으므로, header=None 옵션 지정
- array 함수를 사용하여 데이터 변환



```
data = pd.read_csv('./sample_data/mnist_test.csv',header=None)
data = np.array(data)
```



실습

1. 데이터 읽기
2. 데이터 확인
3. 이미지 데이터와 정답 정보 분할
4. 이미지 데이터의 확인
5. 데이터 차원변경, 수치화
6. 딥러닝 모델 생성
7. 학습
8. 테스트 데이터 읽기
9. 테스트 데이터의 차원변경, 수치화

- pandas와 numpy 라이브러리 사용
- read_csv 함수를 사용하여 csv 파일 읽기
- csv 파일에 열 이름이 따로 없으므로, header=None 옵션 지정
- array 함수를 사용하여 데이터 변환



```
data = pd.read_csv('./sample_data/mnist_test.csv',header=None)
data = np.array(data)
```




실습

1. 데이터 읽기
2. 데이터 확인
3. 이미지 데이터와 정답 정보 분할
4. 이미지 데이터의 확인
5. 데이터 차원변경, 수치화
6. 딥러닝 모델 생성
7. 학습
8. 테스트 데이터의 차원변경

- 이미지 데이터의 확인
 - 테스트 데이터를 $n \times 28 \times 28 \times 1$ 크기로 변경
(`expand_dims` 함수를 쓰지 않고, `reshape` 함수를 사용해 한꺼번에 변경)
- 원-핫 인코딩
 - 테스트 데이터에서, 출력 데이터의 원-핫 인코딩은 불필요
- 결과 확인
 - 바뀐 모양과 값을 코드창에서 확인



```
x_test = data[:,1:]
y_test = data[:,0]
x_test = x_test.reshape(-1,28,28,1)
```



```
x_test.shape
```



```
(10000, 28, 28, 1)
```



```
y_test.shape
```



```
(10000,)
```

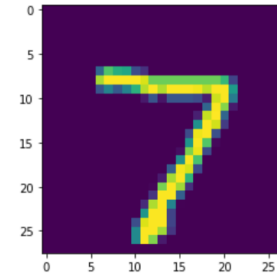


실습

1. 데이터 읽기
2. 데이터 확인
3. 이미지 데이터와 정답 정보 분할
4. 이미지 데이터의 확인
5. 데이터 차원변경, 수치화
6. 딥러닝 모델 생성
7. 학습
8. 테스트 데이터의 차원변경
9. 예측



```
plt.imshow(x_test[0,:].reshape(28,28))
```



```
o = model.predict(x_test)
print(o[0, :]) #첫 번째 테스트 이미지의 결과 확인
```



```
[1.1518536e-23 3.6408084e-30 4.0943083e-18 1.5665343e-19 2.7707018e-29 3.3562561e-24 1.8942200e-34 1.0000000e+00 2.8070306e-23 2.5271057e-22]
```



```
o = np.argmax(o,-1)
print(o[0, :]) #첫 번째 테스트 이미지의 결과 확인
```



```
7
```

- 이미지 데이터의 확인
 - 첫 번째 테스트 이미지 확인
- 예측
 - predict 함수를 사용해 예측
 - 예측된 결과는 원-핫-인코딩 된 상태 (n x 10 형태)
- 10진수 형태로 변경
 - argmax함수를 사용하여 십진수 형태로 변경



실습

1. 데이터 읽기
2. 데이터 확인
3. 이미지 데이터와 정답 정보 분할
4. 이미지 데이터의 확인
5. 데이터 차원변경, 수치화
6. 딥러닝 모델 생성
7. 학습
8. 테스트 데이터의 차원변경
9. 예측
10. 예측 정확도 산출



```
sum(y_test == o)/len(y_test)
```



0.9749

- `y_test == o`
 - 예측한 값(o)과 정답(y_test)이 같은지 여부를 Boolean 배열로 획득
- `sum( Boolean배열 )`
 - 주어진 Boolean 배열 중 True 값의 개수를 계산
- `len(y_test)`
 - 정답 개수 계산
- `sum(y_test == o)/ len(y_test)`
 - 정확도 계산
 - 예측한 값과 정답이 같은 경우를 세어 테스트 데이터의 총 수로 나눔



결과 분석 (1/2)

- 한 번에 완벽한 딥러닝 네트워크를 만드는 것은 불가능
- 자동 파라미터 최적화 툴
 - AutoKeras 등
 - 데이터 종류의 제약 등 존재
- Confusion Matrix
 - 정답(타깃)과 실제 출력을 비교한 표
 - 한 축(세로축)은 타깃, 다른 축(가로축)은 출력
 - 왼쪽 그림에서 1행 2열의 값은
 - 클래스 A의 데이터를 클래스 B로 예측한 수
 - 올바른 예측횟수 = 대각성분의 합
- 정확도
 - $\frac{\text{올바르게 인식한 데이터 수}}{\text{전체 데이터 수}}$
 - 왼쪽 그림에서의 정확도: $270/300 = 90\%$

		출력		
		클래스 A	클래스 B	클래스 C
타깃	클래스 A	90	10	0
	클래스 B	0	90	10
	클래스 C	10	0	90

3개 클래스 분류시의 Confusion Matrix 예



결과 분석 (2/2)

- Confusion Matrix 분석
 - R1, R2 모두 정확도(Accuracy)는 90%로 동일
 - R1의 경우, 에러가 각 클래스별로 골고루 발생
 - R2의 경우, 클래스 A-B사이에서 오인식이 집중적으로 발생
→ 클래스 A-B사이에서 나타나는 문제를 해결하면 인식률의 대폭적인 향상이 기대
 - 전처리, 데이터 수집 방식의 변경 등 딥러닝 외적인 부분에서의 변화도 고려 필요

		출력		
		클래스 A	클래스 B	클래스 C
타깃	클래스 A	90	10	0
	클래스 B	0	90	10
	클래스 C	10	0	90

R1: 3개 클래스 분류시의 Confusion Matrix 예 1 (p.310)

		출력		
		클래스 A	클래스 B	클래스 C
타깃	클래스 A	90	10	0
	클래스 B	20	80	0
	클래스 C	0	0	100

R2: 3개 클래스 분류시의 Confusion Matrix 예 2(p.311)



연습문제 (p.313)

4. 다음 표는 Confusion Matrix라고 불리는 것으로, 숫자들이 각각 어느 숫자로 인식되었는지를 나타낸다.

예를 들어 숫자 5 이미지가 5로 인식되었다면 다음 표에서 (타깃 = 5, 실제 결과 = 5)인 지점에 1을 더하고, (아무것도 기록되지 않은 곳은 0이다.) 숫자 5 데이터가 9로 인식되었다면 (타깃 = 5, 실제 결과 = 9)인 지점의 값에 1을 더한다. 모든 데이터가 올바르게 인식되었다면 대각성분을 제외한 모든 값이 0이 된다.

테스트 결과를 분석하는 코드를 파이썬으로 작성하고, 다음 표를 채우시오.

		실제 결과									
		0	1	2	3	4	5	6	7	8	9
타 깃	0										
	1										
	2										
	3										
	4										
	5										
	6										
	7										
	8										
	9										



연습문제 (p.313) 풀이

4. 다음 표는 Confusion Matrix라고 불리는 것으로, 숫자들이 각각 어느 숫자로 인식되었는지를 나타낸다.

예를 들어 숫자 5 이미지가 5로 인식되었다면 다음 표에서 (타깃 = 5, 실제 결과 = 5)인 지점에 1을 더하고, (아무것도 기록되지 않은 곳은 0이다.) 숫자 5 데이터가 9로 인식되었다면 (타깃 = 5, 실제 결과 = 9)인 지점의 값에 1을 더한다. 모든 데이터가 올바르게 인식되었다면 대각성분을 제외한 모든 값이 0이 된다.

테스트 결과를 분석하는 코드를 파이썬으로 작성하고, 다음 표를 채우시오.



#confusion matrix를 나타내는 변수 생성. 정수형 타입으로 선언
c_matrix = np.zeros([10,10]).astype(np.int)

for i in range(y_test.shape[0]): #모든 테스트 데이터에 대해
 idx_target = y_test[i] #타깃 정보 (세로축)
 idx_output = o[i] #출력 정보 (가로축)

#해당 위치의 c_matrix 값을 1 증가
c_matrix[idx_target, idx_output] += 1

print(c_matrix) #confusion matrix 출력



```
[[ 973   0   0   0   0   2   0   2   1   2]
 [  0 1121   2   5   3   1   0   2   1   0]
 [   1   3 993   9   5   0   2  18   1   0]
 [   0   0   1 997   0   3   0   6   1   2]
 [   3   0   1   0 964   0   1   1   2  10]
 [   3   0   0  20   0 865   1   0   0   3]
 [  12   2   0   0   3   6 935   0   0   0]
 [   0   2   6   5   0   0   0 1006   2   7]
 [   4   0   4  26   5   7   5   8 902  13]
 [   3   3   1   7   7   2   0   9   1 976]]
```

46

감사합니다

THANK YOU